

# Exercise Sheet 1: A first glance at binary code

**Remarks:** These exercises are your homework, and the solutions will be discussed at the exercise on Monday, Oct 27. Although it is not necessary to participate in the exercise or hand in your solutions, contents of the exercise will be relevant for the exam.

## Exercise 1:

**Setup:** In the remainder of the course, we will use some tools that will help us gain insight into binary programs. Install them and make sure they are working.

- (a) We are going to assume you are working in a POSIX-compliant x86\_64 environment.

**(Windows)** Windows users are advised to install *Windows Subsystem for Linux*. WSL2 is available for any up-to-date Windows 10 or 11 installation. You can start the installation by opening a *privileged* command line and execute the command:

```
> wsl --install
```

Because it installs a virtual machine, you may need to enable the corresponding features in your BIOS or EFI. By default, Ubuntu is installed, so you can follow the instructions for Linux in the next parts.

In case you cannot activate virtualization features, you can switch to WSL1 (you might need to enable the Windows Feature “Windows Subsystem for Linux” beforehand):

```
> wsl --set-default-version 1
> wsl --install
```

- (b) Make sure all the required tools are available:

```
$ objdump --version
$ strip --version
$ gcc --version
$ gdb --version
$ file --version
```

**(Windows)** Follow instructions for Linux.

**(Linux)** Install the required packages using your package manager.

```
# Ubuntu
$ sudo apt install build-essential gcc gdb
# Arch
$ sudo pacman -Syu --needed binutils gcc gdb
```

**(Mac)** While you can install the tools on your Mac directly by following the prompt to install *Xcode Command Line Tools* when trying to execute one of the above mentioned programs, we strongly recommend to install a x86\_64 Linux virtual machine. On an ARM

based Mac, this will be emulated instead of virtualized, so it will run slowly if you install a rich graphical environment.

- (c) *Ghidra* is a Reverse Engineering suite developed by NSA, contains a disassembler, decompiler, and much more. To install Ghidra, follow the instructions at <sup>1</sup>. Some Linux distributions, e.g. Arch Linux, offer a package in their repositories.

## Exercise 2:

**Compiling and linking:** In this exercise you will compile a small C program with a library and take a look at the output. Familiarize yourself with the tools using their man page. Look up the purpose of the tools and the meaning of each command line argument used here.

- (a) Download the files `mylib.c` and `myprog.c` from Moodle.
- (b) Compile the library code `mylib.c` to an object file, and create a static library.

```
# Create mylib.o
$ gcc -c mylib.c

# lib$LIBRARYNAME.{a,so} are canonical names for libraries
$ ar -rcs libmylib.a mylib.o

# Inspect the resulting files
$ file libmylib.a
$ objdump -x libmylib.a
$ objdump -d -M intel libmylib.a
```

Compile the main program, and link against the created library:

```
$ gcc -o myprog myprog.c -e my_entry -nostdlib -nostartfiles -lc -L. -lmylib

# Inspect the resulting file
$ file myprog
$ objdump -x myprog
$ objdump -d -M intel myprog
$ ldd myprog
```

Execute the program.

```
# Make the program executable
$ chmod +x myprog

# Execute it
$ ./myprog

# Print the return value (exit code)
$ echo $?
```

---

<sup>1</sup><https://ghidra-sre.org/InstallationGuide.html>

We can also force gcc to link everything statically, even libc. You can now see the implementation of `_exit`.

```
$ gcc -o myprog_static myprog.c -e my_entry -nostdlib -nostartfiles -lc -L.  
-lmylib -static  
  
# Inspect the resulting file  
$ file myprog_static  
$ objdump -x myprog_static  
$ objdump -d -M intel myprog_static  
$ ldd myprog_static
```

- (c) Now, we are going to compile the library code into a shared object file, so that we can use it to link dynamically.

```
# Remove the compiled files  
$ rm *.{a,o,so}  
  
# Create the shared library. Mind the options!  
$ gcc -c mylib.c -fPIC  
$ gcc mylib.o -shared -o libmylib.so  
  
# Inspect the resulting files  
$ file libmylib.so  
$ objdump -x libmylib.so  
$ objdump -d -M intel libmylib.so
```

Compile the main program, and link against the dynamic library:

```
$ gcc -o myprog_dynamic myprog.c -e my_entry -nostdlib -nostartfiles -lc -L.  
-lmylib  
  
# Inspect the resulting file  
$ file myprog_dynamic  
$ objdump -x myprog_dynamic  
$ objdump -d -M intel myprog_dynamic  
$ ldd myprog_dynamic
```

Execute the program.

```
# Make the program executable  
$ chmod +x myprog_dynamic  
  
# Try to execute it  
$ ./myprog_dynamic  
  
# Tell the operating system where to look for the library
```

```
$ export LD_LIBRARY_PATH=.  
$ ./myprog_dynamic  
  
# Print the return value (exit code)  
$ echo $?
```

- (d) Think about advantages and disadvantages of static (compile-time) and dynamic (runtime) linking for the programmer, the user, and the operating system itself.

### Exercise 3:

**Dive deeper:** We will now take a deeper look into our binaries.

- (a) Look up the terms *import table*, *export table*, *global offset table*, *procedure linkage table*, *code section*, and *data section*. Try to find the sections and tables in your binaries created in the previous step.
- (b) Compile different versions of our program with different optimization levels.

```
$ for i in 0 1 2 3; do gcc -o myprog$i myprog.c mylib.c -O$i -e my_entry -  
    nostdlib -nostartfiles -lc; done
```

-O0 implies no optimization, while -O3 tells the compiler to optimize aggressively.

- (c) In the code `myprog.c`, there are two variables defined, `my_var1` and `my_var2`. Try to find them in `myprog0`. Infer a rule about *local* and *global* variables.

**Hint:** Make use of the fact that these variables are assigned hardcoded values.

- (d) Compare the source code with the resulting assembly code in `myprog0`. Try to match the control structures by looking at jump instructions (`jb`, `jmp`), compare instructions (`cmpl`), and call instructions (`call`).

Now, look at the differences in the further optimized code. What high-level changes to the control flow can you observe?